Pick-and-Place Using Inverse Kinematics in MuJoCo: Benchmarking DLS, QP, and Mink Solvers on HEAL and Franka Manipulators

Kush Mehta (24110177)
https://github.com/Kush-Mehta/ME369_Intro_to_Robotics_24110177/tree/main/Lab3

Abstract

This report compares three Inverse Kinematics (IK) solvers—Damped Least Squares (DLS), Quadratic Programming (QP), and Mink—for a pick-and-place task in MuJoCo, tested on a 6-DOF HEAL arm and a 7-DOF Franka Panda arm (100 episodes each). A "Smart Collision" waypoint strategy is used to keep the end-effector on safe vertical paths, so every recorded failure is a pure kinematic reach failure, not a collision. QP is the most reliable solver on both robots (93% HEAL, 92% Franka); DLS is the fastest but violates joint limits most on the Franka arm; Mink is the smoothest but slowest per step.

I. Introduction

Inverse Kinematics (IK) is the process of figuring out *how to move the joints* of a robot arm so that the hand (end-effector) reaches a given target position and orientation. While the forward direction—computing where the hand ends up given joint angles—is straightforward, going backwards is much harder because there may be many possible joint solutions, or none at all, especially near the edges of the robot's reach.

This lab implements and compares three ways of solving IK:

1. **Damped Least Squares (DLS):** A fast, closed-form formula that adds a small damping term to avoid blow-ups near singular (stretched-out) configurations.

2. **Quadratic Programming (QP):** Solves the same problem but as a constrained optimisation, so joint limits are respected automatically rather than as an afterthought.

3. **Mink (baseline):** An off-the-shelf library that combines a QP solver with built-in posture and velocity constraints, used as the reference to compare against.

Two robots are tested:

- **6-DOF HEAL arm** – exactly 6 joints, so there is usually one solution (square Jacobian, $\mathbf{J} \in \mathbb{R}^{6\times6}$).

- **7-DOF Franka Panda** – one extra joint (redundant), giving the solver more flexibility but also more complexity (rectangular Jacobian, $\mathbf{J} \in \mathbb{R}^{6\times7}$).

II. Workspace and Setup

A. Table Size and Why It Was Chosen Deliberately

Both robots use the **same table**, with the cube and placement tray spawning randomly inside these boundaries:

- X-axis: -0.28 m to $+0.28$ m
- Y-axis: $+0.32$ m to $+0.75$ m

The table was deliberately chosen to be large enough to push the robots to their limits—targets near the corners test what happens at the very edge of the arm's reach. Figure 1 shows the Franka arm's full reachable workspace (red point cloud) with the table rectangle (blue) sitting comfortably inside it.

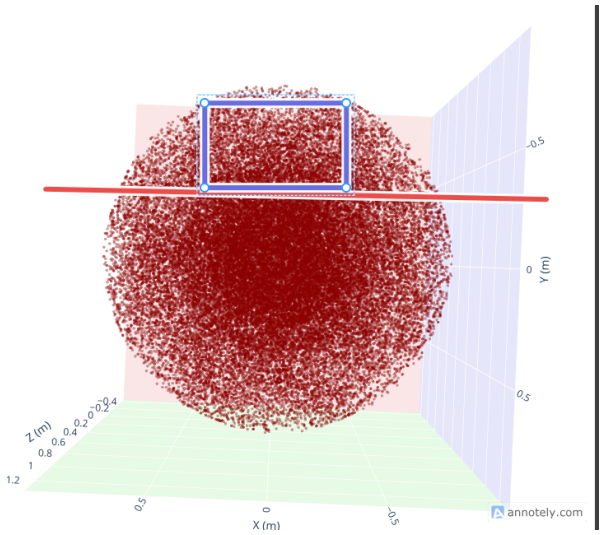


Figure 1: Franka workspace (red cloud) with the table rectangle (blue box) inside it.

B. Why the HEAL Arm Needed a Radial Cap

The same table was first tried for the HEAL arm without any changes. Because the HEAL arm is physically shorter (≈0.66 m reach at grasp height), targets near the table corners were simply out of reach—the arm could not get there no matter which solver was used. Running 100 episodes without any limit gave only a **24% success rate**, and when the failures were analysed, almost all of them were positions outside the arm's reach (Figure **??** shows how failures cluster at the outer edge).

Rather than shrinking the table—which would make the

comparison with the Franka unfair—a **radial cap of** $r_{\max} = 0.63$ **m** was added to the HEAL arm's spawn area. This keeps the same table but limits where the cube can spawn, so every failure that still occurs reflects a real solver limitation, not a hopeless target.

C. How the Workspace Was Generated (Forward Kinematics Sampling)

Before running any IK experiments, the reachable workspace of each robot was mapped out using **forward kinematics (FK) sampling**. The idea is simple: instead of asking "what joint angles reach this point?" (which is IK), we ask the opposite—"if we set the joints to this angle combination, where does the hand end up?" By sweeping through thousands of joint angle combinations, we collect all the hand positions the robot can possibly reach, which together form the robot's *task workspace*.

Algorithm

1. For each joint k, create a grid of N evenly-spaced angles between its minimum and maximum limit: $\theta_k \in [\theta_{k,\min},\ \theta_{k,\max}]$, $\quad N = 8$ samples per joint.
2. Take every combination of joint angles across all joints (i.e. the Cartesian product of all grids). For a 6-DOF robot this gives $8^6 = 262{,}144$ configurations; for 7-DOF it gives $8^7 \approx 2.1$ million.
3. For each configuration $\mathbf{q}$, run MuJoCo's forward kinematics to get the TCP (tool-centre-point) position:

$$\mathbf{p} = \mathrm{FK}(\mathbf{q}) \in \mathbb{R}^3. \tag{1}$$

4. Record all (x, y, z) positions. The resulting point cloud is the task workspace (Figure 1).

A point is also classified as **dexterous** if the robot can reach it with *all* sampled wrist-yaw angles—meaning the hand can spin freely at that location. This is checked by fixing all other joints and sweeping the last joint through $N_{\text{yaw}} = 6$ evenly-spaced values; if the TCP position does not move by more than 1.5 cm, the point counts as dexterous.

Joint Limits Used

The joint limits determine the size and shape of the workspace. They were sourced differently for each robot:

- **HEAL arm:** Limits were read *directly from the robot's MuJoCo XML file* (`arm_gripper.xml`), which is the single source of truth. All 6 joints have symmetric limits of $\pm\pi$ rad (± 180).
- **Franka Panda:** Limits were taken from the **official Franka Emika datasheet** (confirmed in the MuJoCo Menagerie repository), since the minimal simulation XML does not fully encode them.

Table 1 lists the exact limits for both robots.

Table 1: Joint angle limits for both robots (degrees).

Robot	Joint	Min (°)	Max (°)
HEAL (6-DOF)	joint_1	−180	+180
	joint_2	−180	+180
	joint_3	−180	+180
	joint_4	−180	+180
	joint_5	−180	+180
	joint_6	−180	+180
Franka (7-DOF)	joint1 (A1)	−166	+166
	joint2 (A2)	−101	+101
	joint3 (A3)	−166	+166
	joint4 (A4)	−176	−4
	joint5 (A5)	−166	+166
	joint6 (A6)	−1	+215
	joint7 (A7)	−166	+166

HEAL limits from `arm_gripper.xml`; Franka limits from official datasheet.

III. Methods

A. 7-Phase Pick-and-Place Task

The robot completes a pick-and-place in seven steps: Approach → Descend → Grasp → Lift → Transit → Lower → Release. A PD controller inside MuJoCo tracks the joint velocity commands from the IK solver at every control step ($\Delta t_{\text{IK}} = 0.02$ s).

B. Smart Collision Strategy

If the robot moves in a straight line from one pose to another, it can easily clip the table edge or tray wall on the way. To fix this, the robot always moves *vertically* first—up to a safe height, across, then straight down—using fixed height offsets above the target:

$$z_{\text{approach}} = z_{\text{target}} + 0.12 \text{ m}, \tag{2}$$

$$z_{\text{grasp}} = z_{\text{target}} + 0.01 \text{ m}, \tag{3}$$

$$z_{\text{lift}} = z_{\text{target}} + 0.15 \text{ m}. \tag{4}$$

This completely removes sweep-based collisions. Every failure in the logs is therefore a **kinematic infeasibility**—the solver ran out of iterations because the target was physically out of reach, not because it bumped into something.

C. Trajectory Breadcrumbing

Instead of jumping directly to a far target, the path is broken into small intermediate steps at a constant speed of $v = 0.03$ m/s:

$$T_{\text{move}} = \max\left(\frac{D}{v},\ T_{\min}\right), \tag{5}$$

$$N_{\text{steps}} = \left\lfloor \frac{T_{\text{move}}}{\Delta t_{\text{IK}}} \right\rfloor, \tag{6}$$

with linear position interpolation and SLERP for orientation at each fraction $\alpha_i = i/N_{\text{steps}}$:

$$\mathbf{p}_d(\alpha_i) = \mathbf{p}_{\text{start}} + \alpha_i(\mathbf{p}_{\text{target}} - \mathbf{p}_{\text{start}}), \tag{7}$$
$$\mathbf{q}_d(\alpha_i) = \text{SLERP}(\mathbf{q}_{\text{start}}, \mathbf{q}_{\text{target}}, \alpha_i). \tag{8}$$

D. Error Clamping

Before passing the 6D error $\mathbf{e} = [\Delta\mathbf{p}^\top, \Delta\boldsymbol{\theta}^\top]^\top$ to any solver, it is clamped to avoid large jumps:

$$\bar{\mathbf{e}} = \mathbf{e} \cdot \min\left(1,\ \frac{e_{\max}}{\|\mathbf{e}\|_2}\right), \tag{9}$$

where $e_{\max} = 0.05$ m (position) and 0.3 rad (rotation).

E. Damped Least Squares (DLS)

DLS solves for the joint update using:

$$\Delta\mathbf{q}_{\text{task}} = \mathbf{J}^\top\left(\mathbf{J}\mathbf{J}^\top + \lambda^2\mathbf{I}\right)^{-1}\bar{\mathbf{e}}, \quad \lambda = 0.01. \tag{10}$$

The damping term $\lambda^2\mathbf{I}$ prevents the solution from blowing up when the Jacobian is near-singular (arm nearly fully stretched). For the Franka arm, a null-space term keeps the elbow in a stable position:

$$\Delta\mathbf{q} = \Delta\mathbf{q}_{\text{task}} + \left(\mathbf{I} - \mathbf{J}^{+}\mathbf{J}\right) k_{\text{null}}(\mathbf{q}_{\text{def}} - \mathbf{q}), \quad k_{\text{null}} = 0.5. \tag{11}$$

Joint steps are capped at 0.16 rad per iteration.

F. Quadratic Programming (QP)

QP turns IK into a constrained minimisation problem:

$$\min_{\Delta\mathbf{q}} \tfrac{1}{2}\Delta\mathbf{q}^\top\mathbf{P}\,\Delta\mathbf{q} + \mathbf{c}^\top\Delta\mathbf{q}, \tag{12}$$
$$\text{s.t.} \quad \mathbf{lb} \le \Delta\mathbf{q} \le \mathbf{ub}, \tag{13}$$

where $\mathbf{P} = 2(\mathbf{J}^\top\mathbf{J} + 10^{-4}\mathbf{I})$ and $\mathbf{c} = -2\mathbf{J}^\top\bar{\mathbf{e}}$. The bounds $\mathbf{lb}$, $\mathbf{ub}$ are computed to respect both velocity limits and position limits at each step:

$$\text{lb}_k = \max\left(-\dot{q}_{\max},\ \frac{q_{k,\min} - q_k}{\Delta t}\right), \tag{14}$$
$$\text{ub}_k = \min\left(\dot{q}_{\max},\ \frac{q_{k,\max} - q_k}{\Delta t}\right). \tag{15}$$

G. Mink

Mink wraps a QP solver with a velocity-based formulation ($\dot{\mathbf{x}} = \mathbf{J}\dot{\mathbf{q}}$), adding: a tracking task (position weight 1.0, orientation weight 0.8), a posture regularisation task (weight 0.03), and hard velocity bounds at ±8.0 rad/s.

IV. Results

A. HEAL Arm (6-DOF)

Table 2 summarises results on the HEAL arm after applying the 0.63 m radial cap. QP is the most successful (93%), both QP and DLS are fast (∼0.02 s per step), and all solvers record zero joint-limit violations because the 6-DOF arm stays well inside its hardware limits for the sampled poses.

Table 2: HEAL arm results (100 episodes, $r_{\max} = 0.63$ m).

Solver	Success	Time (s)	Iters	JLV
Mink	92%	0.100	191	0
DLS	90%	0.022	325	0
QP	93%	0.021	262	0

JLV = joint-limit violations/episode.

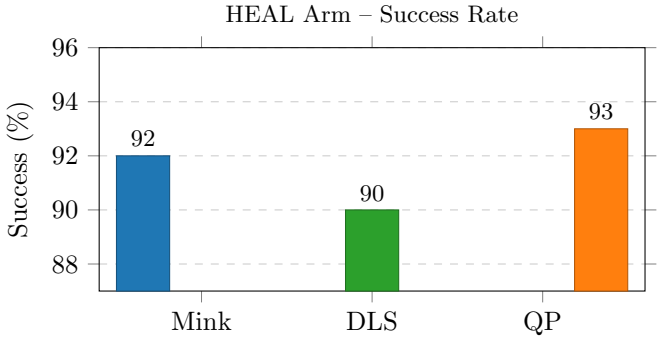


Figure 2: HEAL arm success rates. QP leads; all solvers are above 90%.

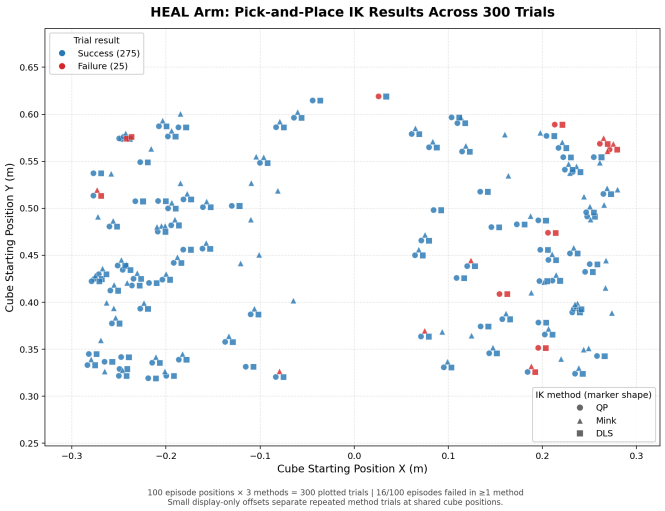


Figure 3: Pick and Place IK results for Heal

B. Franka Panda (7-DOF)

Table 3 shows the Franka results. QP again leads on success rate (92%) and has the fewest joint-limit violations (11.29/episode) because it enforces limits as hard constraints. DLS is the fastest (0.007 s/step) but hits joint limits most often (42.88/episode) since it only clips velocities after the fact. Mink sits in the middle on both metrics.

Table 3: Franka Panda results (100 episodes, no radial cap).

Solver	Success	Time (s)	Iters	JLV
Mink	91%	0.013	96	28.15
DLS	91%	0.007	115	42.88
QP	92%	0.008	108	11.29

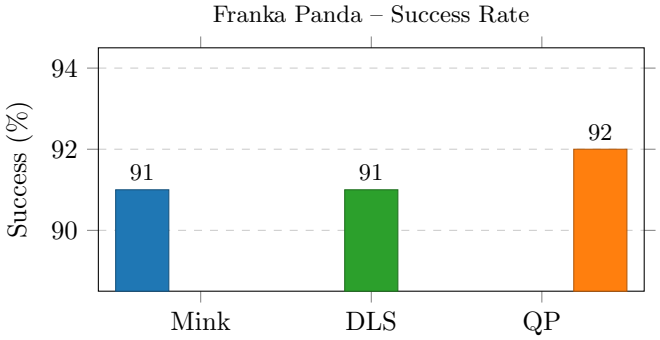


Figure 4: Franka success rates. QP leads at 92%; Mink and DLS tied at 91%.

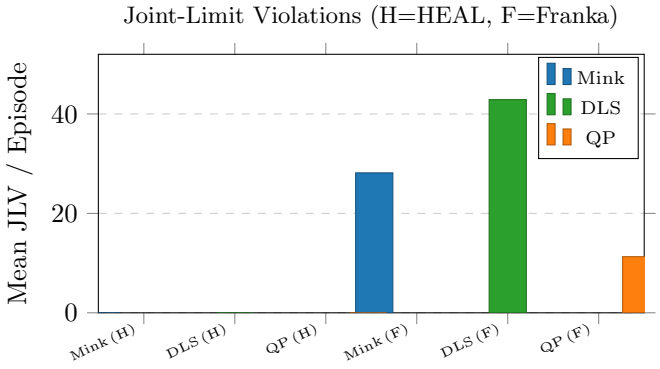


Figure 5: Joint-limit violations per episode. HEAL arm: zero for all solvers. Franka: DLS has most; QP has fewest.

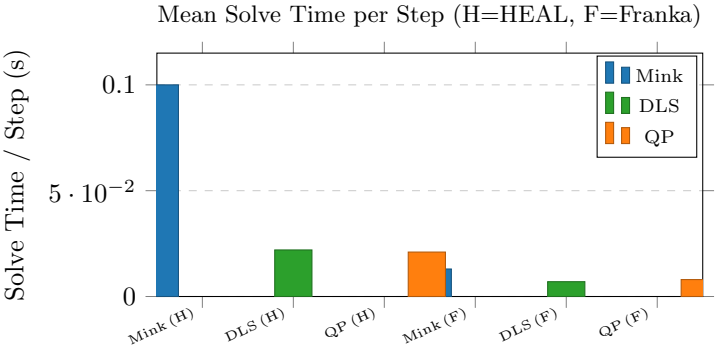


Figure 6: Mean solve time per step. DLS is fastest on both robots. Mink is slow on HEAL but competitive on Franka.

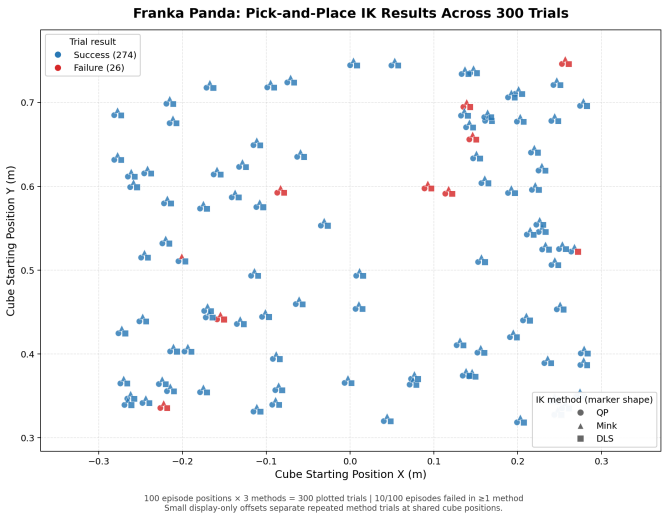


Figure 7: Pick and Place IK results for Heal

V. Failure Analysis

A. What "IK Infeasible" Means Here

When a failure is logged, the solver stalled with a remaining position error of about 0.010–0.018 m after using all 650 allowed iterations. The solver was not diverging or oscillating—it simply ran out of moves because the arm was at its physical limit and could not close the last gap.

B. HEAL Arm: Failures at the Reach Boundary

After the 0.63 m cap, the remaining failures are mostly at targets very close to that boundary. For the QP log (7 failures): episodes 36, 76, and 99 had cube positions at radii 0.630 m, 0.628 m, and 0.629 m—within 2 mm of the cap. The other four failures were at moderate radii where the *tray* position (the destination, not the cube) required an additional stretch that pushed the arm to its limit.

C. Franka Arm: Two Types of Failure

Without a radial cap, the Franka sees two different failure modes in the DLS log (9 failures total):

1. **Wrist singularity (2 failures):** Episodes 1 and 85 had very large orientation errors left over (> 0.3 rad) with 1,336 joint-limit hits each. The arm got to the right position but could not rotate the wrist into the final orientation without crossing a singularity.
2. **Plain reach failure (7 failures):** The remaining episodes had small position errors (0.01–0.02 m) and almost no orientation error—identical to the HEAL boundary failures, just at larger radii.

Table 4: Total failures per solver across both robots.

Solver	HEAL	Franka	Total
Mink	8	9	17
DLS	10	9	19
QP	7	8	15

VI. Discussion

QP vs. DLS trade-off. DLS is about 2× faster per step but ignores joint limits during the solve. On the HEAL arm (non-redundant), this does not matter—limits are never close to being hit. On the Franka arm (redundant, tighter limits), DLS hits joint limits 4× more often than QP per episode. If real-time speed matters, DLS is better for the HEAL arm; QP is safer for the Franka arm.

Mink's overhead vs. benefit. Mink is 5× slower than DLS on the HEAL arm (0.100 vs. 0.022 s/step) but achieves competitive success rates. On the Franka arm its overhead shrinks to only 2×, because its posture task actively steers the null space away from singular configurations, converging in fewer iterations (96 vs. 115 for DLS).

All failures are reach failures. The Smart Collision strategy worked perfectly—no collision-based failures appear anywhere in the logs. This means any future improvement should target the kinematic solver (better singularity handling or workspace-adaptive planning), not the collision strategy.

VII. Conclusion

Three IK solvers were tested on 200 pick-and-place episodes across two robots in MuJoCo. The table was deliberately chosen to be large enough to test the arms at their limits. For the HEAL arm, a 0.63 m radial cap was added after analysis showed uncapped runs produced only 24% success, almost entirely due to out-of-reach targets.

Key takeaways:

- **QP** is the most reliable solver on both platforms (93% HEAL, 92% Franka) with the best joint-limit handling on the redundant Franka arm.
- **DLS** is the fastest solver and works well when joint limits are not a concern (HEAL arm), but accumulates the most joint-limit violations on the Franka arm.
- **Mink** provides the smoothest motion and handles near-singular configurations best, at the cost of being the slowest per step.
- The **Smart Collision** waypoint strategy eliminated all collision-based failures; every remaining failure is a pure kinematic reach limitation.

Recommendation: Use QP when reliability and joint safety matter most; use DLS for speed in open workspaces; use Mink when singularity robustness is the priority.